



Customer 360 & CAR: Trade-offs, Privacy & Stakeholder Q&A

Why the design landed where it did instead of at the next-best alternative, what a regulator or Sharia board would actually ask about it, and where the platform is deliberately still exposed.

PART 2 OF 3 — TRADE-OFFS, PRIVACY & STAKEHOLDER Q&A

Prepared For	Companion Documents	Status	Scope
Engineering, Product/Exec, AI/ML, Compliance & Sharia Governance	#1 Architecture ERD #2 #3 Identity SQL	Draft for review	Trade-off rationale, privacy/regulatory posture, cross-cutting risk register

Contents

How to Read This Document	2
1 Design Trade-offs Revisited	2
1.1 Why AWS Glue FindMatches, and What Would Change That Answer	2
1.2 Why MD5 Over a Generated UUID for the ECID	3
1.3 Why Redshift and RDS Instead of Picking One	3
1.4 Why SCD Type 2 Instead of Event Sourcing or Bitemporal Modeling	3
1.5 Why Batch, Not Streaming, for the CAR	4
1.6 What Happens When a Merge Is Wrong	4
2 Privacy & Regulatory Posture	5
2.1 Consent, Access, Erasure – and What Actually Conflicts With What	5
2.2 One Nuance Worth Flagging: Credit Data May Sit Outside the PDPL	6
2.3 Sharia Governance: Who Actually Reviews This Platform	6
3 AI/ML Team Q&A	7
4 Product & Executive Q&A	8
5 Cross-Cutting Risk Register	9



How to Read This Document

Deliverables 1 through 3 answered *what* the platform does and *how* it is built. This document answers a different question: *what happens when someone with a stake in the outcome pushes back on it?* It assumes the reader has already approved the architecture and is now asking the harder follow-up questions – an engineer wondering why a specific technology was picked over its obvious competitor, a regulator or the Sharia board asking whether the design actually satisfies a specific legal obligation, an AI/ML lead asking where the platform could quietly introduce bias, or an executive asking what happens if a Day 60 milestone slips.

Each section is organized around one stakeholder’s actual concerns, not around the platform’s own structure – which means the same underlying design decision sometimes gets revisited from two different angles in two different sections, deliberately, because a compliance officer and a Redshift engineer are never asking about the same trade-off for the same reason. Questions are written the way a genuinely skeptical stakeholder would ask them, not softened into something easy to answer.

Where a claim rests on an external regulation rather than an internal design choice – UAE data protection law, Central Bank record-keeping rules, Sharia governance structure – this document cites the specific source rather than asserting it from memory, because getting that wrong here would be worse than getting it wrong anywhere else in the set.

1 Design Trade-offs Revisited

This section is written for the audience most likely to have built something differently – other data engineers. Every trade-off below was already touched on in Deliverable 1; what follows is the version of that discussion where the next-best alternative gets a fair hearing instead of a passing mention.

1.1 Why AWS Glue FindMatches, and What Would Change That Answer

AN ENGINEERING PEER ASKS

“You picked a managed AWS black box for the single highest-stakes decision in the platform – who gets merged into who. Doesn’t that bother you?”

It should bother anyone evaluating this design, and it’s worth being precise about what “black box” means here rather than accepting the label. FindMatches is opaque in one specific way: it doesn’t expose the learned feature weights the way a hand-rolled Fellegi–Sunter implementation would. It is not opaque in the ways that actually matter for a bank – every match it produces carries a `match_confidence` score and a `match_method` tag (Deliverable 3, Section 2), every match is reproducible from the same inputs, and the entire training set is inspectable because it’s built from Mal’s own labeled pairs, not a vendor-supplied model.

The honest trade-off is operational, not epistemic: Zingg gives up nothing on inspectability and would answer “why did the model think these matched” more completely, at the cost of standing up and owning a Spark deployment Mal doesn’t otherwise need in Q2. That is a real cost, not a rounding error, which is why Zingg stays named as the specific fallback in Deliverable 1 rather than being dismissed. The trigger for revisiting this decision is concrete and already written down: the first time Compliance asks for feature-level match explainability that FindMatches genuinely cannot provide, this decision is reopened, not defended.



1.2 Why MD5 Over a Generated UUID for the ECID

AN ENGINEERING PEER ASKS

“MD5 is a broken hash for anything security-sensitive. Why is it anywhere near a core identifier?”

Because nothing here asks MD5 to do a security job. `ecid = MD5(national_id_token)` is not a cryptographic commitment – the token underneath it is already the HMAC-SHA256 output doing the actual security work (Deliverable 3, Section 1). MD5 is doing exactly one thing: deterministically collapsing a 64-character token into a shorter, stable, idempotent identifier, so that resolving the same national ID token twice – on a replay, a backfill, or a re-run after a failure – always produces the same ECID without a lookup table in the loop. A random UUID could not do that; it would need a separate keyed lookup (token → UUID) that becomes one more table that can drift, be double-written, or race under concurrent inserts.

The trade-off this doesn't get for free is collision resistance under adversarial input, which is irrelevant here since the input space is Mal's own already-tokenized customer base, not attacker-supplied data – and birthday-bound collision probability at even ten million rows is astronomically below the platform's actual failure modes (a bad merge from a matching error is millions of times more likely than an MD5 collision). If this were ever exposed to adversarially-chosen input, or if 128 bits of hash space stopped being comfortably sufficient at scale, SHA-256 truncated to the same length is a drop-in replacement with no schema change – which is precisely why this is flagged as a reasoned choice in a rationale box rather than left unstated.

1.3 Why Redshift and RDS Instead of Picking One

AN ENGINEERING PEER ASKS

“Two stores for one golden table sounds like the kind of thing that looks fine in a design doc and turns into a consistency nightmare in production.”

That risk is real and is exactly why the design constrains it to a single direction: RDS is read-only, populated exclusively by the same nightly SCD2 publish job that updates Redshift (Deliverable 1, Section 3), and nothing – no application, no ad hoc script – is permitted to write to the RDS copy directly. That constraint is what makes “two stores never disagree” a property of the pipeline rather than a hope. The failure mode that constraint doesn't eliminate is staleness *during* a publish window: if the nightly job partially completes, Redshift and RDS could briefly disagree about what “current” means for a customer who resolved that same night. Deliverable 1 doesn't currently name a specific mitigation for that window, and it should – a parity-check job comparing row counts and `scd_valid_from` timestamps across both stores after every publish is a reasonable Day 90 hardening item and is added to the risk register in Section 5.

The alternative – Redshift alone, with the mobile app and support console hitting it directly – was rejected for a narrower reason than “Redshift is slow”: it's specifically the tail latency of a single-row point lookup against a columnar, MPP-distributed table that a support agent pulling up one customer's profile would feel, not the platform's aggregate query throughput.

1.4 Why SCD Type 2 Instead of Event Sourcing or Bitemporal Modeling

AN ENGINEERING PEER ASKS

“SCD2 is a 1990s dimensional-modeling pattern. Why not something that actually models time properly, like a bitemporal table or full event sourcing?”

Because the question SCD2 answers – “what did we believe about this customer on a given date” – is the one this platform is actually asked, repeatedly, for audit and for keeping model training free of label leakage (Deliverable 2, Section 1). A bitemporal model would additionally answer “when did we *learn* that fact,” which matters when a correction needs to be distinguished from a genuine change – and that distinction has a real use here: a survivorship correction after a bad merge is exactly a case where knowing *when Mal found out* would matter, not just what changed. That's a legitimate gap, not a hypothetical one, and it's the honest answer to why this decision could reasonably go the other way. It wasn't made because bitemporal modeling is unusual in banking data warehouses; it's genuinely more common there than in most industries. It was made because



Mal doesn't yet have a concrete downstream consumer asking the "when did we learn" question, and every one of those SCD2 records is going to be read far more often by Redshift's query planner than reasoned about by a data engineer, which favors the simpler, better-optimized, better-understood pattern until a real requirement forces the more expressive one.

Full event sourcing was never seriously in contention for the golden layer specifically because the golden record's job is to answer "what is true right now, and what was true then" cheaply – event sourcing is the right tool when the sequence of changes is itself the object of interest, which describes `IDENTITY_CROSSWALK`'s merge/split history far better than it describes `CUSTOMER_GOLDEN`'s current state. That's a genuine architectural fork worth naming out loud: the crosswalk is already closer to an event log than a dimension, and a later engineer proposing to model it more explicitly as one would not be wrong.

1.5 Why Batch, Not Streaming, for the CAR

AN ENGINEERING PEER ASKS

"A fraud model refreshed once a day is a fraud model that's a day behind a fraudster. Why does this design accept that?"

It doesn't accept it as a permanent state – it accepts it as a Q2 state, deliberately, because a real-time fraud signal built on an identity layer that hasn't been proven yet is a real-time fraud signal built on sand. The sequencing in Deliverable 1's 30/60/90 plan is a hard dependency, not a preference: identity resolution has to be trustworthy before the golden 360 means anything, the golden 360 has to exist before the CAR can be built on top of it, and only once both are proven does a minutes-scale refresh for the genuinely time-sensitive features become a design worth doing carefully rather than quickly. Building the streaming layer first and retrofitting identity resolution underneath it would mean re-deriving every fraud-relevant feature the moment a merge gets corrected – exactly backwards from the platform's stated invariant that identity resolution stays upstream of everything else (Deliverable 2's cardinality rationale).

What this trade-off actually costs Mal in the meantime is named explicitly rather than hidden: the 24-hour freshness commitment is a ceiling on the CAR as a whole, and Deliverable 1 already flags that select fraud-relevant features are the specific candidates for a minutes-scale path once the streaming layer exists – which is the subject of Part 3.

1.6 What Happens When a Merge Is Wrong

AN ENGINEERING PEER ASKS

"Every identity resolution system eventually merges the wrong two people. Walk me through what actually happens operationally when that occurs at Mal, not just the theory."

Mechanically: a steward or an automated check (Deliverable 3, Section 5's Check 1) flags an ECID with more than one distinct `national_id_token` behind it – the one invariant this entire design treats as worse than a missed match. Fixing it is a crosswalk correction, not a golden-record rewrite: the offending `identity_crosswalk` row is re-pointed to a new or different ECID, which by construction changes which rows the SCD2 survivorship view considers when it next runs, and the next nightly publish closes out the old golden record's current-version flag and opens a corrected one.

What the theory glosses over and the practice has to answer is *everything already built on top of the wrong ECID* in the meantime – CAR snapshots already published under the merged identity, model features already computed and potentially used in a live decision, and a support agent who may have already spoken to the wrong "customer" about the wrong account. None of those are automatically corrected by a crosswalk fix, and Deliverable 1 doesn't currently specify a remediation runbook for the downstream blast radius. That gap is real and is called out again in Section 5's risk register rather than papered over here.



2 Privacy & Regulatory Posture

This section is written for the audience that will actually be asked to sign off on the platform before it touches a real customer – Compliance, the data protection function, and Mal’s Internal Sharia Supervision Committee (ISSC). Every regulatory claim below is cited to its source rather than asserted from general banking-industry familiarity, because that distinction matters more here than anywhere else in this document set.

2.1 Consent, Access, Erasure – and What Actually Conflicts With What

COMPLIANCE ASKS

“Federal Decree-Law No. 45 of 2021 (the UAE PDPL) gives customers a right to erasure. Your design keeps years of SCD2 history and an immutable Bronze layer. Which one of you is wrong?”

Neither, but the tension is real and worth stating precisely instead of arguing it away. The PDPL’s consent article (Article 6) requires that consent be obtained in a clear, simple, and accessible way, and that withdrawal be easy and *not retroactively invalidate processing that already happened lawfully* – which is exactly the semantics the CONSENT entity already implements: `granted_at` and `withdrawn_at` are both recorded, nothing is deleted, and the nightly CAR job simply stops populating gated columns going forward once `withdrawn_at` is set (Deliverable 1, Section 4; Deliverable 2, Section 1). That part of the design and that part of the law are already aligned.

The part that is not automatically aligned is a hard erasure request against `CUSTOMER_GOLDEN` or `stg_core_customer` history, because Central Bank of the UAE record-keeping rules require licensed financial institutions to retain customer identification and transaction records for five years from the end of the business relationship, not zero.^a A literal delete-on-request would put Mal in violation of one obligation to satisfy another. The resolution this design already leans toward, without yet stating it as a formal policy, is the standard one: an erasure request is honored as a re-tokenization and access lockout – the customer’s PII tokens are rotated to non-reversible values and their token-vault entry is purged, which makes every historical record permanently non-re-identifiable without touching the retained row count a regulator would come looking for. Deliverable 1 does not currently name this policy explicitly, and it should before this design is presented to Compliance as erasure-capable.

^aCBUAE Rulebook, Record Retention (AML/CFT), §6.4 – <https://rulebook.centralbank.ae/en/rulebook/64-record-retention>.

COMPLIANCE ASKS

“Where does the data physically live, and does that create a cross-border transfer problem under the PDPL?”

AWS operates a UAE region (`me-central-1`, launched in Abu Dhabi and Dubai in 2022),^a and every service named in this platform’s stack – S3, Glue, Redshift, RDS, Lake Formation – is available there, so the default and intended posture is that customer PII never leaves the UAE in the first place. That sidesteps the PDPL’s cross-border transfer requirements (Article 22, in force whenever data does leave to a jurisdiction without an adequacy decision) rather than satisfying them, which is a meaningfully different and stronger claim, but it is only true if every future integration – a SaaS analytics tool, a third-party fraud vendor, an AI model provider – is vetted against that constraint explicitly. Nothing in the current design enforces that automatically; it is a governance process this document is flagging as necessary, not a technical control that exists yet.

^aAWS News Blog, *Now Open – AWS Region in the United Arab Emirates* – <https://aws.amazon.com/blogs/aws/now-open-aws-region-in-the-united-arab-emirates-uae/>.

COMPLIANCE ASKS

“What is Mal’s obligation if this platform is ever breached?”



Immediate notification to the UAE Data Office and to affected individuals, describing the nature of the breach, its likely effects, and the remediation already taken.^a “Immediately” is the statute’s word, not a negotiable internal SLA, which has a direct design consequence: the PII tier model’s audit logging (Deliverable 1, Section 4) has to be good enough to answer *which* customers were affected and *which* tier of access was involved within hours, not after a forensic investigation concludes weeks later. That’s an argument for treating Tier 2 and Tier 3 access logs as a first-class, continuously monitored data product in their own right, not a byproduct of Lake Formation’s default grants – which is not yet how Deliverable 1 frames them.

^aUAE PDPL (Federal Decree-Law No. 45 of 2021), breach notification obligations; summarized in Securiti, *Overview of UAE’s Federal Decree-Law No. (45) of 2021* – <https://securiti.ai/uae-personal-data-protection-law/>.

2.2 One Nuance Worth Flagging: Credit Data May Sit Outside the PDPL

WHY THIS WAY: The PDPL explicitly excludes “health or credit data governed by their own respective legislation” from its own scope (Article 2(2)).^a In practice, that most plausibly points to credit-bureau-sourced data – if Mal ever ingests a credit report or bureau score from Al Etihad Credit Bureau into the CAR’s risk features, that specific data may be governed by separate credit-reporting rules rather than the PDPL obligations described above, with a potentially different consent and retention regime attached. This document does not attempt to resolve that question, because it depends on a data-sourcing decision Deliverable 1 doesn’t currently make – it’s flagged here so Compliance evaluates it before, not after, a bureau feed gets added to the CAR’s risk-features group.

^aIbid.

2.3 Sharia Governance: Who Actually Reviews This Platform

SHARIA BOARD (ISSC) ASKS

“This whole document talks about data governance. Where does Sharia governance actually touch any of this?”

At exactly one deliberate point in the schema, and everywhere that point’s consequences propagate, `PRODUCT.shariah_contract_type` carries the actual underlying contract structure – Murabaha, Mudarabah, Wakala – instead of an interest rate, because that is the field Product and Compliance teams, and by extension the ISSC, actually need to reason about (Deliverable 2, Section 1). Under CBUAE’s Shari’ah governance standard, Mal’s ISSC is bound by the fatwas and standards set by the Central Bank’s Higher Shari’ah Authority (HSA), which holds supervisory authority over every licensed Islamic financial institution’s internal committee and can conduct its own external audit of a bank’s operations at that bank’s expense.^a That means the ISSC’s actual, practical question about this platform is not abstract: can it prove, on demand and for any account, that no interest-bearing computation exists anywhere in the pipeline, and that every product a customer holds resolves cleanly to a contract type the ISSC has approved?

The honest answer today is that the schema supports this proof but the platform doesn’t yet automate it. `shariah_contract_type` being a required, typed field on every `PRODUCT` row means the query “list every account whose product has no approved contract type” is trivial to write and should be a standing, ISSC-visible check rather than an ad hoc audit – which is a concrete, low-cost addition worth making before this platform is presented to the ISSC for sign-off, not after.

^aCBUAE Rulebook, Article (17): Higher Shari’ah Authority – <https://rulebook.centralbank.ae/en/rulebook/article-17-higher-shari%E2%80%99ah-authority>.



3 AI/ML Team Q&A

AI/ML LEAD ASKS

“Jaro-Winkler similarity was trained and tuned on Western naming conventions. Amina’s example resolves cleanly on a phone number, so we never see the hard case in Deliverable 1 – what happens to a customer whose name is genuinely ambiguous under this metric?”

This is the most concrete fairness risk in the entire matching design, and it’s worth naming plainly: Arabic full names carry structural ambiguity that Jaro-Winkler was never designed around – multiple valid transliterations of the same name (Mohammed / Muhammad / Mohamed), patronymic chains that get truncated inconsistently across systems, and the “Al” family-name prefix that shifts where the meaningful comparison substring even starts. A CRM lead whose name transliterates differently across two systems is measurably more likely to land in the steward review band, or worse, below it, than a customer whose name has one unambiguous spelling – which means the population most likely to need Pass 4 (Deliverable 1, Section 2.2) may not be a random cross-section of Mal’s customers.

The design’s actual defense against this today is structural, not statistical: nothing in the middle or lower confidence band auto-merges (Deliverable 3, Section 3), so a name-similarity miscalibration produces more steward review work and more unmatched residuals, not more wrong merges – the failure mode this design is built to avoid above all others stays avoided even under this bias. But “produces more manual review for one population than another” is itself a fairness problem worth tracking, not just a cost problem. This should be an explicit Day 60 monitoring item: track auto-merge rate, steward-queue rate, and unmatched rate segmented by name origin or script, not just in aggregate, and revisit the similarity function – an Arabic-aware phonetic algorithm, or name-part-weighted comparison instead of a flat string metric – if the segmented numbers show what this question predicts they will.

AI/ML LEAD ASKS

“How do you actually stop label leakage in model training, concretely – not the one-line answer in Deliverable 2?”

SCD2’s `scd_valid_from / scd_valid_to` pair makes “what did we know about this customer as of date *D*” a queryable fact rather than an approximation, which is what closes the most common leakage path: training a model on a customer’s *current* risk rating or KYC status when the label being predicted is itself dated months earlier, silently teaching the model to see the future. Building a training set correctly here means joining to `dim_customer_golden` AS OF the label date using the SCD2 window, not joining to the current row – which is a query discipline the platform makes possible but does not enforce. A feature-store-style point-in-time join helper, built once and reused by every model team, is the concrete artifact that turns “SCD2 prevents leakage” from a true statement about the schema into a true statement about what actually gets trained.

AI/ML LEAD ASKS

“What’s actually stopping someone on my team from re-identifying a customer through the tokenized fields, if they’re determined to?”

Structurally, the token-vault lookup – the only reversible path back to a raw identifier – sits behind Tier 3, requiring named-individual approval, dual sign-off, and an independently audited log entry per lookup (Deliverable 1, Section 4). An AI/ML engineer’s default Tier 1 access gives a stable, joinable pseudonym and nothing that reverses. What that doesn’t stop, and what tokenization never stops, is re-identification by *correlation* rather than reversal: an engineer with Tier 1 access to enough joined columns – a rare product combination, a specific transaction pattern, a small residency-country cohort – can sometimes narrow a pseudonym down to one plausible real person without ever touching the token vault. That risk is inherent to any sufficiently rich, joinable dataset and isn’t unique to this design, but it’s also not addressed by anything currently in Deliverable 1’s tier model, which controls *column* access but not query-level re-identification risk from combining many columns at once. A k-anonymity or minimum-cohort-size check on any dataset extracted for offline model development is the standard mitigation and is worth adding as an explicit Tier 1 data-export control, not just a column grant.



AI/ML LEAD ASKS

“You ruled out an LLM-based agent for the merge decision. Is there anywhere in this pipeline an agent genuinely earns its place, beyond the one narrow triage-assistant mention in Deliverable 1?”

One place, concretely, and it’s worth being specific about the guardrails rather than leaving it as a vague possibility: summarizing *why* a candidate pair landed in the steward review queue – rendering the Fellegi–Sunter feature comparison (name similarity score, DOB match, blocking key) as a plain-language explanation a steward reads before making their own decision. The guardrail that keeps this from becoming a second, informal decision-maker is that the agent would never see or influence `match_confidence` or `match_method` – those are already computed before the agent runs, and the steward’s decision, not the agent’s summary, is what gets logged as the actual merge action. An agent that could alter those fields, or whose output got treated as a recommendation to accept or reject, would reintroduce exactly the unaccountable judgment call Deliverable 1 explicitly designed the whole waterfall to avoid. This stays explicitly out of scope for Q2, and any future proposal to expand an agent’s role here should be evaluated against that same standard: does it explain a decision that a human still makes, or does it make the decision.

4 Product & Executive Q&A

AN EXECUTIVE ASKS

“Every piece of this stack is AWS-native. What happens to Mal if AWS pricing changes, or if we need to move to another cloud in three years?”

The honest framing is that this design accepts vendor concentration risk deliberately in exchange for shipping in Q2 without also standing up and operating a second platform’s worth of infrastructure – that trade was made explicitly in Deliverable 1’s technology comparison, not by default. What limits the damage of that concentration is where the actual data lives: Bronze and Silver both sit in S3 as source-shaped and Parquet-friendly files under a Glue Catalog (Deliverable 1, Section 3), which are portable formats readable by any engine, not a proprietary AWS structure. The genuinely hard-to-move pieces are Redshift’s star schema and any FindMatches-trained model artifacts, both of which would need to be rebuilt, not just copied, on another platform. A pragmatic mitigation worth adopting now, at effectively no cost, is keeping the Glue Schema Registry contracts (Deliverable 1, Section 6) as the actual source of truth for schema shape, stored as reviewable files in the ETL repo rather than only inside AWS’s console – so the schema itself survives a future migration even if the compute underneath it doesn’t.

AN EXECUTIVE ASKS

“What happens to the roadmap if FindMatches turns out to perform badly once we see real Day 60 data?”

The platform doesn’t stop functioning – it degrades gracefully to something already true today rather than something new that has to be built under pressure. Passes 1 through 3 are entirely independent of FindMatches and already resolve the overwhelming majority of real matches (Deliverable 1, Section 2.2); a poorly-performing Pass 4 simply means a larger residual pool sits in the steward review queue or goes unmatched a while longer, not that the golden Customer 360 becomes wrong. The fallback path named in Deliverable 1 – the hand-rolled Jaro-Winkler threshold originally scoped as a Day 30 stopgap – can be extended a while longer as a conservative bridge while either FindMatches is retrained on a larger seed set or Zingg is evaluated as the alternative already shortlisted for exactly this situation. The actual executive-level risk isn’t a broken platform; it’s a slower-than-planned reduction in the size of the manual steward review queue, which is a staffing and timeline cost, not an architectural failure.

AN EXECUTIVE ASKS

“Of everything deliberately deferred to Year 2 – household modeling, joint accounts, real-time streaming – which one is most likely to actually hurt the business if it slips past Year 2?”



Household and relationship modeling, for a reason specific to fraud rather than personalization: fraud rings are disproportionately family- or household-structured, and the longer that modeling stays out of scope, the longer a specific, known class of coordinated fraud stays outside what the platform can detect at all – not degraded, structurally invisible, because the schema has no relationship edge between two golden customer records to even query (Deliverable 1, Section 3’s rationale). Joint accounts and real-time streaming both degrade gracefully in their absence – Mal simply doesn’t yet serve joint-account customers or sub-daily fraud signals, which is a product gap, not a blind spot. Household modeling’s absence is different in kind: it’s not a feature Mal hasn’t built yet, it’s a fraud vector Mal cannot currently see. That’s worth the business explicitly accepting as a known risk, in writing, rather than it being an implicit consequence of a scoping decision made for good reasons at the time.

AN EXECUTIVE ASKS

“Where is the single most likely place the 30/60/90 timeline actually slips?”

Day 60, specifically the FindMatches labeling step, and the reason is structural rather than a scheduling guess: it’s the one milestone in the entire plan that depends on a business process (KYC operations stewards reviewing sample pairs) rather than an engineering task Mal fully controls the pace of (Deliverable 1, Section 2.2’s feasibility discussion). Every other Day 30 and Day 60 deliverable is gated by engineering effort; this one is gated by a different team’s bandwidth and by how ambiguous the actual residual pool of CRM-only leads turns out to be once real data is behind it. This is also, not coincidentally, the one Day 60 item that can slip without blocking the Day 30 launch’s critical path, since the deterministic waterfall it depends on already stands alone – which is the argument for treating a Day 60 slip on this specific item as an accepted, bounded risk rather than a reason to hold the broader platform back.

5 Cross-Cutting Risk Register

Every risk below traces back to a specific gap surfaced in Sections 1 through 4 – this table exists so the platform’s known, accepted exposures live in one place instead of being scattered across four stakeholder conversations.



Risk	Likelihood	Impact	Mitigation
Two different customers merged into one ECID	Low	Critical	Conservative Fellegi–Sunter thresholds; Check 1 validation query run every batch; crosswalk-only correction path (Section 1.6)
Genuine duplicate never resolves (false negative)	Medium	Medium	Day 60 probabilistic pass; unmatched pool re-scored on every run, never silently discarded
FindMatches underperforms on a small/biased Day 60 seed set	Medium	Medium	Bootstrap labels from deterministic matches; steward feedback loop retrains calibration over time (Section 4)
Name-similarity metric under-serves non-Western naming conventions	Medium	Medium	Nothing auto-merges below threshold; add name-origin-segmented monitoring at Day 60 (Section 3)
Token vault is a single point of re-identification failure	Low	Critical	Tier 3 dual-approval break-glass; independently audited per lookup; KMS customer-managed key
Redshift/RDS golden-layer drift during a publish window	Low	Medium	RDS is read-only, single publish source; add a post-publish parity check as a Day 90 item (Section 1.3)
Data-quality gate false-blocks a valid CAR publish	Medium	Low	Last-known-good snapshot served instead; on-call paged; thresholds tuned during Day 60 hardening
Consent-withdrawal lag lets a gated feature compute post-withdrawal	Low	High	Nightly consent check before populating gated columns; explicit Day 60 consent-withdrawal batch job
AWS-native stack limits future portability	High (by design)	Low–Medium	Open formats (S3/Parquet) for Bronze/Silver; schema contracts kept as reviewable files outside the console
Day 60 FindMatches labeling milestone slips	Medium	Medium	Deterministic waterfall alone already covers the majority of matches; does not block Day 30 launch critical path
Household/relationship fraud stays structurally undetectable	Medium	High	Explicitly accepted as a known Year 1 gap (Section 4); design begins Day 90 per Deliverable 1's execution plan

This Document's Place

This document assumes Deliverables 1 through 3 as its factual baseline and does not restate their content – it stress-tests it. The two gaps this document surfaces that Deliverable 1 does not yet name explicitly – a formal erasure-as-re-tokenization policy (Section 2.1) and a post-publish Redshift/RDS parity check (Section 1.3) – are candidates for a future revision of that document rather than something resolved here. **Part 3** of this engagement takes up real-time streaming: the minutes-scale refresh path for fraud-relevant CAR features that Section 1.5 of this document explains why Mal deliberately isn't building yet.